

Proposal for Orbital 26

Team Name: MarkPals

Proposed Level of Achievement: Artemis

Motivation

Studying for exams involves grinding past year papers, but marking them manually takes time and feels repetitive if we are doing multiple papers.

Aim

We hope to create an application that allows us to save the time and effort of marking our past year papers by grading them automatically for us.

Full Discretion:

We understand the risks of using AIs and LLMs and their possible tendency to hallucinate. Therefore, we urge users to double-check the responses of features involving LLMs (such as the grading of open-ended questions).

Our core goal is for this app to be a tool to assist users in saving time through auto-marking, not as a replacement for critical-thinking capabilities.

User Stories

1. As a student who wants to study for my midterms, I want to be able to study more by reducing the time and effort spent on marking my own papers.
2. As a student who wants to save time, I want to be able to mark my past year papers automatically.

Features

1. Feature 1 (core): Marking Stack
 - a. Provide users with an answer sheet with open-ended fields to standardise the areas that the OCR needs to scan.
 - b. Receive answer sheet and answer keys from Frontend to Backend
 - c. Use OCR to read answer key and answer sheet on the Backend
 - d. Use LLM to classify questions between MCQ and open-ended questions.
 - e. For MCQ: Compare letter answer for each question to answer key
 - f. For Open-ended/FITB questions: Identify key points from the answer sheet by using LLM to extract and identify main points from the user's answers. We then check whether the user's answers contain the key points in the answer key.
2. Feature 2 (core): Web User Interface
 - a. UI to send files to the backend
 - b. Display the marked paper and the user's marks onto the frontend.
 - c. Allow users to download the marked paper
3. Feature 3 (core): Mobile User Interface
 - a. UI to send files to the backend
 - b. Display the marked paper and the user's marks onto the frontend.
 - c. Allow users to download the marked paper
4. Feature 4 (extension): Tooltip System:
 - a. Hovering over a question shows a tooltip that explains why the user got marked down instead of on the side (allows users to visually relate an answer to a question instead of mentally remembering).
5. Feature 5 (extension): Allow users to upload their own answer sheets
 - a. Originally, users would have to submit the answer sheet provided in point 1(a)
 - b. With this feature, we're planning for users to be able to submit answers without the answer sheet (e.g. answers on a foolscap paper).
6. Feature 6 (extension): Google Drive integration
 - a. Allow users to upload documents directly from Google Drive
7. Feature 7 (extension): Open question/answer in Claude or ChatGPT directly
 - a. This allows users to have full context-appropriate conversations for any follow-up questions after the explanation provided in 4(a)
8. Feature 8 (extension): Running Marking Stack locally
 - a. OCR locally on the mobile app
 - b. LLM locally on the mobile app

Timeline

1. Milestone 1 - Technical proof of concept (i.e., a minimal working system with both the frontend and the backend integrated for a very simple feature)
 - a. Feature 1 done to level 1(c).
 - b. Feature 2 done to level 2(b)
2. Milestone 2 - Prototype (i.e., a working system with the core features)
 - a. Feature 1 done to level 1(f).
 - b. Feature 2 done to level 2(c)
 - c. Feature 3 done to level 3(c)
 - d. Feature 4 done
3. Milestone 3 - Extended system (i.e., a working system with both the core + extension features)
 - a. Feature 5 done
 - b. Feature 6 done
 - c. Feature 7 done
 - d. Feature 8 done

Tech Stack

1. React for Web Interface
2. React Native for Mobile Interface
3. Express.js for Backend
4. OCRmyPDF and/or Tesseract for Optical Character Recognition.
5. ChatGPT, Gemini or Claude API for parts of the Marking Stack
6. GitHub for Version Control and Project Management

Qualifications

Technical Skills:

Frontend: React, React Native, Android Studio, HTML, jQuery

Backend: Node.js, Express, Flask, Java Beans

Database: PostgreSQL, MySQL, Sequelize, MongoDB, Amazon S3

Prototyping: Figma for prototyping and user A/B testing, Miro for Class Diagrams

Project Experience:

1. An Android app that allows users to sleep on trains without worrying about missing their station. Our app allows them to select a station and alerts them when they are nearby. Many train passengers listen to music too, prompting us to integrate Spotify into the application. Developed using Android Studio and Google Maps API.
2. We created a Reddit clone with fully functional subreddits, posts, a voting system and media support to give users another platform to socialise on. This project put our full-stack development capabilities on full display, showcasing the skills we have learnt throughout a Diploma in Information Technology. Developed using jQuery, Express.js, MySQL. Large medias like pictures and gifs were stored in Amazon S3.
3. Interning at Qlassroom gave us the opportunity to work on new features for their website and develop mobile applications with features belonging onto the company's web interface. This allows students and teachers of the platform to access the learning platform on their mobile devices while on the go. We also have experience implementing a local LLM onto the mobile device to recognise digits for one of the app features, which might come in handy for one of the feature extensions. Developed the website using React (frontend) and Express + Postgresql (backend + database), mobile app using React Native.

Software Engineering

- AGILE framework to set clear goals/tasks for each bi-weekly sprint.
 - With each goal being set an importance value and man-hours to decide priority.
- Implementing the Scrum Board within GitHub to list deliverables and delegate work.
- GitHub for version control and pull requests from development branches to vet changes.
 - including standardised commit names/descriptions
- Design class diagrams in MIRO before starting so that developers are on the same page.
- Likewise, we'll prototype the user interface on Figma before starting interface development.
- Use a Model-View-Controller pattern to create a 'separation of concerns' between components to make the application scalable and maintainable.
- CI/CD for automatic checks that our code builds and runs after changes to the codebase.